

Introduction à Spark(RDD)

Classe : Master en Réseaux et système informatique (ISI)

Prof : M. Abdoulaye Barro

Sommaire

1. Introduction à Spark
 1. Définition
 2. Initialisation de Spark
 1. Les options de Spark-shell
 2. SparkContext
2. RDD & DataSet / DataFrame

Introduction à Spark

- **Définition**

- Apache Spark est un système de traitement distribué open source utilisé pour les traitements de données à grande échelle.
- Utilise, pour des requêtes rapides sur des données de toute taille, des processus comme :
 - **In-Memory Caching** : Traitement et Stockage des données intermédiaires en utilisant la mémoire (écriture en Mémoire plus rapide que l'écriture en Disk)
 - **Shuffle** : Mécanisme de re-Distribution/ re-Partition des données lors des opérations.

Introduction à Spark

- **Les composants de Spark**

Apache Spark Core

Spark SQL

Spark
Streaming

MLlib
(Machine
Learning)

GraphX
(Graph)

Introduction à Spark

● Initialisation

● Spark avec

- Scala => **spark-shell** [Options]
- Python => **pyspark** [Options]

● Les Options de lancement

- | | |
|--------------------------------|---|
| ● --name NAME | :name = nom de l'application |
| ● --master TYPE | :type = yarn, local, mesos...etc. |
| ● --deploy-mode MODE | :mode = cluster, client |
| ● --jars JARS | :jars = jars embarqués à l'exécution |
| ● --files FILES | :files = fichiers embarqués à l'exécution |
| ● --driver-memory MEM | :mem = Driver memory (default 1G) |
| ● --num-executors NUM | :num = nombre d'executeurs à l'exécution |
| ● --executor-memory MEM | :mem = executor memory (default 1G) |

- **--queue** QUEUE_NAME :queue_name = nom de la queue utilisée

Exercice: Initialisation Spark

- **Ouvrez une session spark-shell avec les options suivantes :**
 - **Donner comme nom à l'application : 'Spark by « mon_nom »'**
 - **Ouvrez le avec le mode Yarn et le déploiement doit se faire en mode cluster**
 - **5 exécuteurs et chaque exécuteur aura une mémoire de 3G et 5 cores**
 - **Le driver doit avoir une mémoire de 4G**
 - **Vérifier au niveau de l'environnement Yarn, que toutes vos options ont été prise en compte.**

Introduction à Spark

- **Initialisation**

- **SparkContext**

- Point d'entrée principal d'une exécution Spark (Main)
- A l'initialisation d'une Session (ex: spark-shell), l'objet SparkSession est disponible comme avec le nom **spark**
 - Mais peut être réinitialisé :

```
import org.apache.spark.sql.SparkSession
val spark:SparkSession = SparkSession.builder()
    .master("yarn")
    .appName("mon application")
    .getOrCreate()
```

Exercice: Initialisation Spark

- **Dans la session ouverte tout à l'heure :**
 - **Réinitialisée la et changer le nom par 'Change Spark by « mon_nom »'**
 - Avec la methode **getAll** de spark.conf, afficher toutes les options
 - Avec la méthode **get(« key »)**, afficher le nom de votre application. Vous pourrez retrouver la clé au niveau des informations affichées avec la methode **getAll**

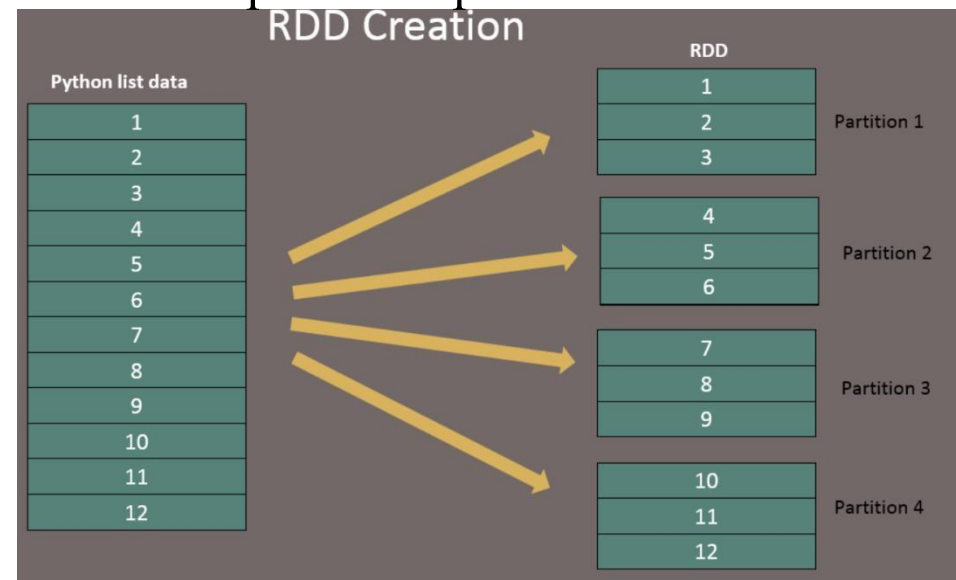
RDD & DataSet / DataFrame

● Les RDDs : Définition

- Resilient Distributed Dataset, sont des « *collections* » d'éléments partitionnées et distribuées à travers les nœuds du cluster
- C'est la structure fondamentale de base dans Spark
- Les RDD permettent d'effectuer des opérations parallèles en mémoire sur un cluster.

● Quelques caractéristiques

- Traitement en mémoire
- Lazy evaluation
- Immuables
- ...



RDD & DataSet / DataFrame

- **Les RDDs : Création**

- Il existe deux (2) manières de créer un RDD

- A partir d'une **collection** :

`spark.sparkContext.parallelize(collection)`

Exemple:

```
val fruits= Seq((1,"orange" ),(2,"pomme"),(3,"fraise"),(4,"citron"))
```

```
val fruitRDD = spark.sparkContext.parallelize(fruits )
```

- A partir d'une **source externe** :

`spark.sparkContext.textFile("cheminVersFichier")`

RDD & DataSet / DataFrame

- **Les RDDs : Opérations**

- De manière générale, les rdds supportent 2 types d'opérations:

- **1. Les Transformations :**

A partir d'un RDD, retourne un autre RDD après un traitement

- `map(func)`
- `filter(func)`
- `flatMap(func)`
- `groupByKey([numPartitions])`
- `reduceByKey(func, [numPartitions])`
- `sortByKey([ascending], [numPartitions])`
- `join(otherDataset, [numPartitions ])`
- `coalesce(numPartitions)`
- `repartition(numPartitions)`

RDD & DataSet / DataFrame

- **Les RDDs : Opérations**

- De manière générale, les rdds supportent 2 types d'opérations:

- **2. Les Actions :**

A partir d'un RDD retourne une valeur

- **reduce**(*func*)
- **collect**()
- **count**()
- **first**()
- **countByKey**()
- **foreach**(*func*)

Exercice RDDs

- Ouvrez une session spark
- Créer un RDD en se basant sur le fichier titanic.txt qui se trouve dans votre répertoire de base
- Supprimer la première ligne (entête)
- Créer un RDD contenant les TicketClass distincts
- Calculer le nombre de survivant par TicketClass
- Créer un RDD, ne contenant que les informations de Sex et Survived
- Quel est le nombre de femmes qui ont survécus dans titanic ?
- Créer un RDD ne contenant que les femmes.
- Le max age des survivants et des morts ?
- Quel est le nombre de ligne de titanic.txt ?
- Calculer le nombre de survivants et de morts en one-shot?
- Avec une combinaison de map et reduce, calculer l'âge total des personnes.

FIN
